

Silo

Arquitetura do banco de dados

Modelo de dados do Silo em SQLite: diagrama de relacionamentos, tabelas com campos, tipos, chaves e índices, conteúdo das colunas JSON, máquina de estados, concorrência entre processos, migrações, arquivos em disco, consultas, backup e como levar o modelo para outro banco.

Copyright (©) 2026 Clara Portes

Versão 1.0.0 · Licença MIT

1. Visão geral

O Silo guarda o estado de imagens, containers, portas e eventos num único arquivo SQLite, `silo.db`, dentro de `SILO_ROOT` (padrão `/var/lib/silo`). O mesmo arquivo é lido e escrito por vários processos ao mesmo tempo: a CLI, um supervisor (`silo shim`) por container e a API (`silo serve`).

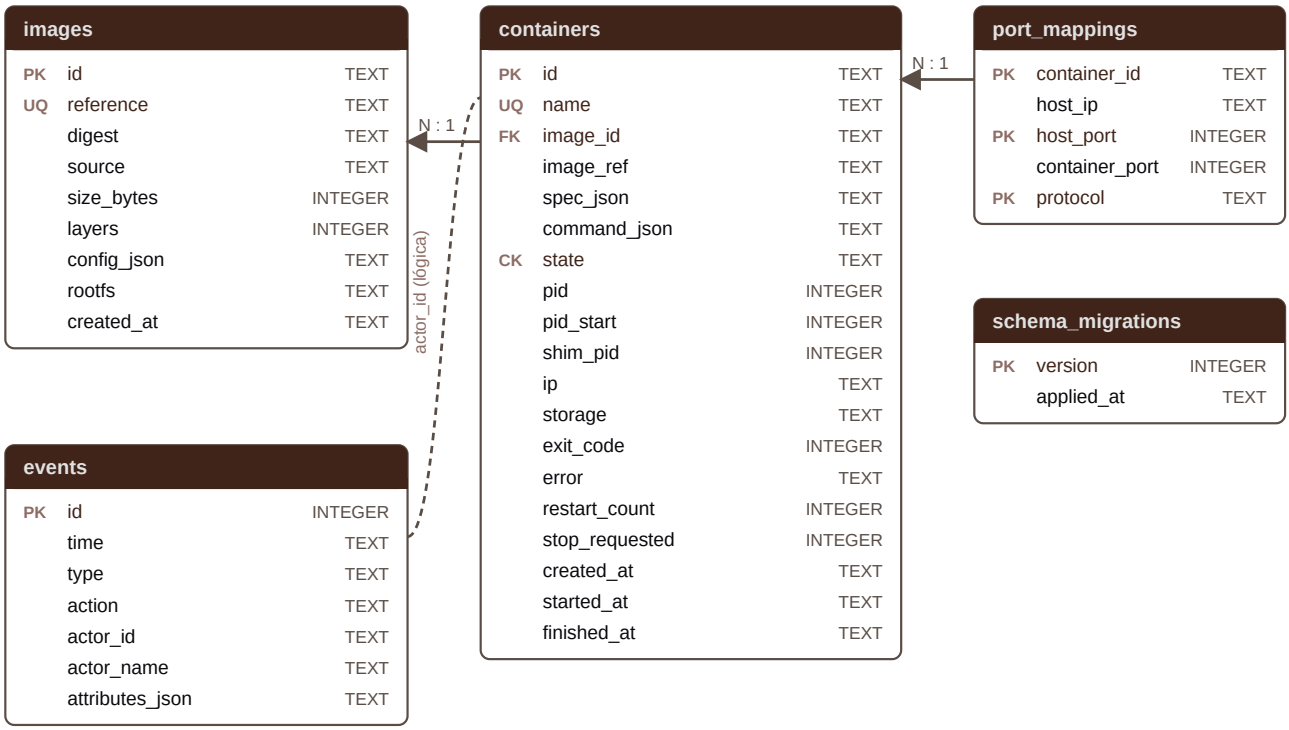
Por que SQLite

- **Embutido:** não há servidor para instalar. O runtime funciona logo depois de copiar o binário.
- **Um host:** containers são locais à máquina, então o estado também é local.
- **Vários processos:** com WAL, leitores não bloqueiam o escritor e cada processo abre a sua conexão.
- **Transações e chaves estrangeiras:** nome único, imagem em uso e portas são garantidos pelo próprio banco.

Parâmetros da conexão

Parâmetro	Valor	Motivo
Driver	`github.com/mattn/go-sqlite3`	SQLite 3 completo via cgo
`_journal_mode`	`WAL`	Leituras simultâneas enquanto um shim grava o estado
`_busy_timeout`	`10000` ms	Espera a trava em vez de falhar quando dois processos gravam juntos
`_foreign_keys`	`on`	Ativa `RESTRICT` e `CASCADE`
`_synchronous`	`NORMAL`	Seguro com WAL e mais rápido que `FULL`
Conexões por processo	até 4	A API atende várias requisições em paralelo

Diagrama de relacionamentos



Uma imagem tem vários containers; um container tem várias portas publicadas. Os eventos não têm chave estrangeira: guardam o ID e o nome do alvo para continuar legíveis depois que o container ou a imagem forem removidos.

2. Tabelas

images

Uma linha por imagem local, baixada de um registro ou importada.

Campo	Tipo	Nulo	Padrão	Regra	Descrição
id	TEXT	não		PK	SHA-256 de referência + digest (64 caracteres hexadecimais)
reference	TEXT	não		UNIQUE	Nome normalizado, ex.: `alpine:3.20`, `ghcr.io/org/app:1.0`
digest	TEXT	não	''		Digest do manifesto (registro) ou do tar (importação)
source	TEXT	não	'registry'		`registry` ou `import`
size_bytes	INTEGER	não	0		Tamanho do rootfs extraído
layers	INTEGER	não	0		Quantidade de camadas aplicadas
config_json	TEXT	não	'{}'	JSON	Configuração da imagem (seção 3)
rootfs	TEXT	não			Caminho absoluto de `images/<id>/rootfs`
created_at	TEXT	não		RFC 3339 UTC	Quando a imagem foi gravada

containers

Uma linha por container, do `create` ao `rm`.

Campo	Tipo	Nulo	Padrão	Regra	Descrição
id	TEXT	não		PK	64 caracteres hexadecimais aleatórios; a CLI mostra os 12 primeiros
name	TEXT	não		UNIQUE	Letras, números, `_ . -`, até 63 caracteres
image_id	TEXT	não		FK → images.id RESTRICT	Impede remover uma imagem em uso
image_ref	TEXT	não			Cópia do nome da imagem para exibição
spec_json	TEXT	não		JSON	Especificação completa pedida pelo usuário (seção 3)

Campo	Tipo	Nulo	Padrão	Regra	Descrição
command_json	TEXT	não	'[]'	JSON	Comando final: entrypoint + cmd já resolvidos
state	TEXT	não		CHECK	`created`, `running`, `stopping` ou `exited`
pid	INTEGER	não	0		PID do processo principal visto pelo host; 0 quando parado
pid_start	INTEGER	não	0		Campo 22 de `/proc/PID/stat`; evita confundir com outro processo que reutilizou o PID
shim_pid	INTEGER	não	0		PID do supervisor
ip	TEXT	não	''		Último IP na bridge
storage	TEXT	não	'overlay'		`overlay` ou `copy`
exit_code	INTEGER	não	0		Código de saída; 128 + sinal quando morto por sinal; 125 a 127 para falhas do Silo
error	TEXT	não	''		Última mensagem de erro (falha no execve, OOM, sem supervisor)
restart_count	INTEGER	não	0		Reinícios pela política
stop_requested	INTEGER	não	0	0 ou 1	Marcado por stop, kill TERM/KILL ou rm; impede a política de reiniciar
created_at, started_at, finished_at	TEXT	started/finished sim		RFC 3339 UTC	Linha do tempo

port_mappings

Portas publicadas de cada container.

Campo	Tipo	Nulo	Padrão	Regra	Descrição
container_id	TEXT	não		PK, FK → containers.id CASCADE	Some junto com o container
host_ip	TEXT	não	'0.0.0.0'		Interface do host
host_port	INTEGER	não		PK, CHECK 1–65535	Porta no host
container_port	INTEGER	não		CHECK 1–65535	Porta dentro do container

Campo	Tipo	Nulo	Padrão	Regra	Descrição
protocol	TEXT	não	'tcp'	PK, CHECK tcp/udp	Protocolo

events

Histórico append-only consumido pela CLI (`silo events`) e pelo WebSocket do painel.

Campo	Tipo	Nulo	Padrão	Regra	Descrição
id	INTEGER	não		PK AUTOINCREMENT	Sequência usada como cursor (`after`)
time	TEXT	não		RFC 3339 UTC	Momento do evento
type	TEXT	não			`container` ou `image`
action	TEXT	não			create, start, stop, kill, die, oom, restart, exec, error, destroy, port-error, pull.start, pull, pull.error, import, delete
actor_id, actor_name	TEXT	não	''		ID e nome do alvo
attributes_json	TEXT	não	'{}'	JSON	Detalhes: exitCode, pid, ip, signal, cmd, digest, error

schema_migrations

Campo	Tipo	Nulo	Padrão	Regra	Descrição
version	INTEGER	não		PK	Número da migração aplicada
applied_at	TEXT	não		RFC 3339 UTC	Quando foi aplicada

Índices

Índice	Tabela e colunas	Usado por
`sqlite_autoindex_images_1`	images(reference)	Busca por nome no `run` , `pull` , `rmi`
`sqlite_autoindex_containers_1`	containers(name)	Busca por nome em todos os comandos
`idx_containers_state`	containers(state)	`ps` , reconciliação, IPs e portas em uso
`idx_containers_image`	containers(image_id)	Checagem de imagem em uso
`idx_ports_host`	port_mappings(host_port, protocol)	Detecção de porta ocupada

Índice	Tabela e colunas	Usado por
`idx_events_actor`	events(actor_id)	Eventos de um container

3. Colunas JSON

Configurações que mudam com as versões ficam em JSON para não exigir migração a cada opção nova. Campos ausentes assumem o valor zero do Go.

containers.spec_json

```
{
  "image": "nginx:alpine",
  "entrypoint": ["/docker-entrypoint.sh"],
  "cmd": ["nginx", "-g", "daemon off;"],
  "env": ["TZ=America/Sao_Paulo"],
  "workingDir": "/",
  "hostname": "web01",
  "user": "101:101",
  "privileged": false,
  "network": "bridge",
  "ports": [{"hostIp": "0.0.0.0", "hostPort": 8080, "containerPort": 80, "protocol":
"tcp"}],
  "mounts": [{"source": "/srv/site", "target": "/usr/share/nginx/html", "readOnly": true}],
  "resources": {"cpus": 0.5, "cpuWeight": 0, "memoryMb": 128, "pidsLimit": 64},
  "restart": {"name": "on-failure", "maxRetries": 3},
  "readOnlyRootfs": false,
  "autoRemove": false,
  "labels": {"equipe": "loja"},
  "dns": ["1.1.1.1"]
}
```

As portas também são gravadas em `port_mappings` para consultas rápidas de conflito. A fonte da verdade para iniciar o container é o `spec_json`.

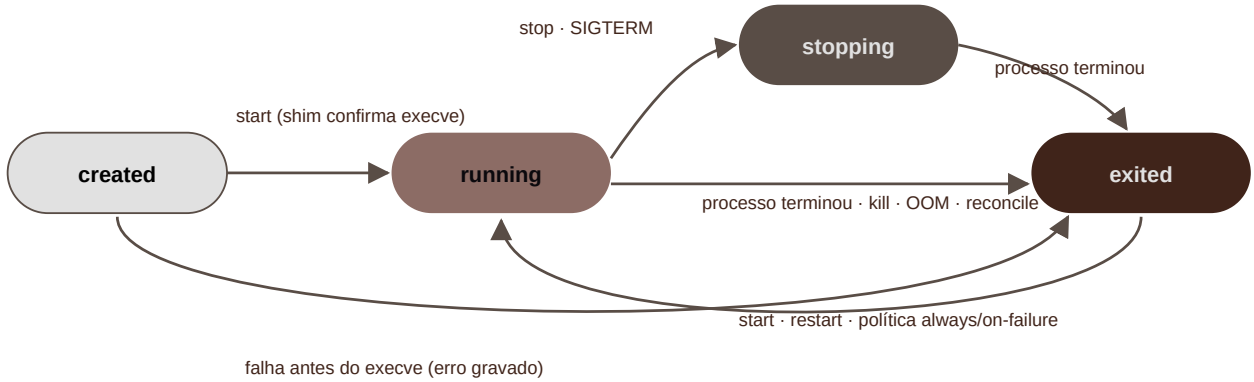
images.config_json

```
{
  "env": ["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"],
  "entrypoint": null,
  "cmd": ["/bin/sh"],
  "workingDir": "",
  "user": "",
  "labels": {"org.opencontainers.image.source": "..."},
  "architecture": "amd64",
  "os": "linux"
}
```

events.attributes_json

Ação	Atributos
start	pid, ip
die	exitCode, oom (quando houver), reason=orphan na reconciliação
kill	signal
exec	cmd
restart	attempt
error, pull.error, port-error	error (e port)
create	image
pull, import	digest

4. Máquina de estados



De	Para	Quem grava	Quando
(nenhum)	created	CLI ou API	`create` ou início do `run`
created / exited	running	shim	Depois que o `execve` deu certo; grava pid, pid_start, shim_pid, ip, started_at e limpa error e finished_at
created / exited	exited	shim	Falha antes do `execve` (imagem ausente, cgroup, rede, comando inexistente); exit_code 125–127 e error
running	stopping	CLI ou API	`stop` ; também marca stop_requested
running / stopping	exited	shim	O processo terminou; grava exit_code, finished_at e zera pid
running / stopping	exited	reconciliação	Processo e shim não existem mais (reinício da máquina, shim morto); exit_code 255
exited	running	shim	Política `always` ou `on-failure` sem stop_requested; incrementa restart_count
qualquer	(removido)	CLI, API ou shim	`rm` , `prune` ou `--rm` ; apaga a linha e, em cascata, as portas

Reconciliação

Todo comando da CLI e a API (a cada 5 segundos) chamam `Reconcile()` : para cada container em `running` ou `stopping` , confere se o PID ainda existe, se não é zumbi e se o `pid_start` bate com `/proc/PID/stat` . Se nem o processo nem o shim estão vivos, o container passa para `exited` , e o cgroup e o veth que sobraram são removidos.

5. Concorrência e integridade

Situação	Como o banco resolve
Dois containers com o mesmo nome	`UNIQUE(name)`; a API responde 409
Remover imagem em uso	`FOREIGN KEY ... ON DELETE RESTRICT` e checagem prévia com mensagem clara
Remover container com portas	`ON DELETE CASCADE` em `port_mappings`
Dois containers rodando na mesma porta	Consulta em `port_mappings JOIN containers` filtrando `running` / `stopping` antes de iniciar
Dois shims pegando o mesmo IP	Trava exclusiva `flock` em `network.lock` desde a escolha do IP até gravar `running`
Vários processos gravando	WAL + `busy_timeout`; cada escrita é curta (uma linha)
Criar container e portas	Mesma transação: ou grava tudo ou nada
PID reutilizado pelo sistema	`pid_start` compara o horário de início do processo

Migrações

As migrações ficam na lista `migrations` em `internal/store/store.go`. Ao abrir o banco, o Silo lê `MAX(version)` de `schema_migrations` e aplica as que faltam, cada uma na sua transação.

1. Acrescente um novo item no fim da lista (nunca altere os anteriores).
2. Use `ALTER TABLE ... ADD COLUMN` com `DEFAULT` para não quebrar linhas existentes.
3. Atualize `containerCols` / `imageCols`, o `scan` e o `INSERT` / `UPDATE` correspondentes.
4. Rode `go test ./internal/store`.

```
`ALTER TABLE containers ADD COLUMN health TEXT NOT NULL DEFAULT '';
 CREATE INDEX idx_containers_health ON containers(health);`,
```

Retenção de eventos

A cada 100 eventos o Silo apaga os que estão além de `SILO_EVENTS_RETAIN` (padrão 5000). O `AUTOINCREMENT` garante que os IDs nunca voltam, então o cursor do WebSocket continua válido.

6. Arquivos em disco

O banco guarda metadados. O conteúdo fica em pastas com o mesmo ID das linhas:

Caminho	Relacionado a	Conteúdo
`silo.db`, `silo.db-wal`, `silo.db-shm`	todas as tabelas	Banco e arquivos do WAL
`images/<images.id>/rootfs/`	`images.rootfs`	Sistema de arquivos extraído
`containers/<containers.id>/upper`, `work`, `rootfs`	`containers.storage`	Camada gravável do overlayfs ou cópia completa
`containers/<id>/container.log`	`containers.id`	Saída do processo
`containers/<id>/shim.log`	`containers.shim_pid`	Mensagens do supervisor
`containers/<id>/hostname`, `hosts`, `resolv.conf`	`spec_json`, `ip`	Arquivos montados no `/etc` do container
`network.lock`	`containers.ip`	Trava da alocação de IP
`/sys/fs/cgroup/silo/<id>` (v2) ou `/sys/fs/cgroup/<controlador>/silo/<id>` (v1)	`containers.id`	Limites e estatísticas; existe só enquanto o container roda

Consultas úteis

```
sudo sqlite3 /var/lib/silo/silo.db

SELECT name, state, exit_code, ip, json_extract(spec_json, '$.resources.memoryMb') AS mem_mb
FROM containers ORDER BY created_at DESC;

SELECT c.name, p.host_port, p.container_port, p.protocol
FROM port_mappings p JOIN containers c ON c.id = p.container_id WHERE c.state = 'running';

SELECT i.reference, COUNT(c.id) AS containers, i.size_bytes / 1048576 AS mb
FROM images i LEFT JOIN containers c ON c.image_id = i.id GROUP BY i.id;

SELECT time, action, actor_name, attributes_json FROM events
WHERE action IN ('die','oom','error') ORDER BY id DESC LIMIT 20;
```

Backup e restauração

```
sudo sqlite3 /var/lib/silo/silo.db ".backup '/root/silo-$(date +%F).db'"
sudo tar -C /var/lib/silo -czf /root/silo-imagens.tar.gz images
```

Para restaurar, pare os containers e o serviço, copie o arquivo de volta para `silo.db` (apagando `silo.db-wal` e `silo.db-shm`) e restaure a pasta `images`. Containers que estavam rodando aparecem como `exited` na primeira reconciliação.

7. Levando para outro banco

Não é necessário para usar o Silo. Só faz sentido se você quiser centralizar o estado de vários hosts num servidor, por exemplo para um painel único. O modelo usa SQL padrão e passa para o PostgreSQL com poucas trocas de tipo:

```
CREATE TABLE images (
  id          CHAR(64) PRIMARY KEY,
  reference   TEXT NOT NULL UNIQUE,
  digest      TEXT NOT NULL DEFAULT '',
  source      TEXT NOT NULL DEFAULT 'registry' CHECK (source IN ('registry','import')),
  size_bytes  BIGINT NOT NULL DEFAULT 0,
  layers      INTEGER NOT NULL DEFAULT 0,
  config_json JSONB NOT NULL DEFAULT '{}',
  rootfs      TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL
);
CREATE TABLE containers (
  id             CHAR(64) PRIMARY KEY,
  host           TEXT NOT NULL,
  name           TEXT NOT NULL,
  image_id       CHAR(64) NOT NULL REFERENCES images(id) ON DELETE RESTRICT,
  image_ref      TEXT NOT NULL,
  spec_json      JSONB NOT NULL,
  command_json   JSONB NOT NULL DEFAULT '[]',
  state          TEXT NOT NULL CHECK (state IN ('created','running','stopping','exited')),
  pid            INTEGER NOT NULL DEFAULT 0,
  pid_start      BIGINT NOT NULL DEFAULT 0,
  shim_pid       INTEGER NOT NULL DEFAULT 0,
  ip             INET,
  storage        TEXT NOT NULL DEFAULT 'overlay',
  exit_code      INTEGER NOT NULL DEFAULT 0,
  error          TEXT NOT NULL DEFAULT '',
  restart_count  INTEGER NOT NULL DEFAULT 0,
  stop_requested BOOLEAN NOT NULL DEFAULT false,
  created_at     TIMESTAMPTZ NOT NULL,
  started_at     TIMESTAMPTZ,
  finished_at    TIMESTAMPTZ,
  UNIQUE (host, name)
);
CREATE TABLE port_mappings (
  container_id   CHAR(64) NOT NULL REFERENCES containers(id) ON DELETE CASCADE,
  host_ip        INET NOT NULL DEFAULT '0.0.0.0',
  host_port      INTEGER NOT NULL CHECK (host_port BETWEEN 1 AND 65535),
  container_port INTEGER NOT NULL CHECK (container_port BETWEEN 1 AND 65535),
  protocol       TEXT NOT NULL DEFAULT 'tcp' CHECK (protocol IN ('tcp','udp')),
  PRIMARY KEY (container_id, host_port, protocol)
);
CREATE TABLE events (
  id              BIGSERIAL PRIMARY KEY,
  host            TEXT NOT NULL,
  time            TIMESTAMPTZ NOT NULL,
  type            TEXT NOT NULL,
  action          TEXT NOT NULL,
  actor_id        TEXT NOT NULL DEFAULT '',
  actor_name      TEXT NOT NULL DEFAULT '',
  attributes_json JSONB NOT NULL DEFAULT '{}'
);
CREATE INDEX idx_containers_state ON containers(state);
CREATE INDEX idx_events_actor ON events(actor_id);
```

> Em vários hosts, inclua a coluna `host` (como acima) e mantenha o `flock` local para IPs, porque cada host tem a sua própria bridge. O `pid`, o `shim_pid` e os caminhos só fazem sentido no host de origem.

Onde trocar no código

1. Em `internal/store/store.go`, troque o driver em `Open` (por exemplo `github.com/jackc/pgx/v5/stdlib`) e os placeholders `?` por `$1, $2, ...`.
2. Troque as migrações pelo DDL acima.
3. Ajuste `isUnique` para o código de erro do novo banco (`23505` no PostgreSQL).
4. O resto do Silo usa só os métodos de `Store` e não muda.